AI 英语朋友

英语老师 · 英国朋友 · 心理委员 三位一体的 AI 英语学习伙伴

技 术 文 档

第九届传智杯 · AI 创新应用挑战赛（个人赛）

作者：李添恩

马鞍山学院 · 软件工程

2026 年 10 月

摘要

本项目「AI 英语朋友」是一个面向英语学习者的智能陪伴式学习应用。它打破了传统英语工具「只会改错、不懂情感」的单一形态，将三种角色——英语老师、英国朋友、心理委员——融合在同一个对话助手当中：既能严谨地纠正语法、出题讲解，也能像朋友一样自然地聊天，还能在用户情绪低落时温柔地倾听与陪伴。应用由 DeepSeek 大模型驱动，界面设计与部分功能在 GitHub Copilot 的协同下完成，体现了「大一新生 + AI 协作」从零到一完整构建一个可用产品的过程。

本文档系统介绍项目的设计目标、功能架构、核心技术实现、Prompt 工程、AI 协同开发过程以及测试与部署方案，作为参赛作品的技术说明。

一、项目概述

1.1 项目背景与动机

很多英语学习者在练习中存在两个痛点：一是「学得孤独」——传统的语法检查工具冷冰冰，只有对错没有温度；二是「学得断裂」——改错、翻译、背单词、做题分散在不同的 App 里，难以形成闭环。作者作为一名大一新生，同样在备考四级的路上遇到这些困扰，于是萌生了「做一个懂我的英语朋友」的想法。

借助 DeepSeek 大模型的自然语言能力，以及 GitHub Copilot 在代码与界面设计上的辅助，作者在短时间内完成了这个从想法到可用产品的完整过程。

1.2 设计目标

- 三位一体：让同一个 AI 根据场景自动切换「老师 / 朋友 / 心理委员」三种身份。
- 学习闭环：从「发现错误 → 出题巩固 → 批改 → 错题重做」形成完整的正反馈循环。
- 个性化：通过轻量规则分析用户的语言习惯与情绪，让 AI 越聊越懂用户。
- 低门槛：一条 API Key 即可运行，核心功能零额外服务依赖。

二、功能设计

2.1 七大界面

界面	核心功能	亮点
💬 **AI 对话**	主界面：角色扮演、语音输入、AI	三位一体身份切换

	头像、个性化画像	
✏ **英语助手**	语法改错 / 中英翻译 / 逐句讲解 / 查单词	改错后可一键闭环出题
📝 **做题本**	AI 围绕知识点出填空题 → 作答 → 批改	批改结果自动解析
❌ **错题本**	错题自动收集、重做、攻克后移除	自动去重
📅 **学习日历**	坚持天数、四级倒计时、月历打卡	里程碑徽章激励
📜 **历史记录**	总练习 / 今日练习 / 待复习错题统计	成长数字可视化
📋 **学习计划**	内置 Daylight 风格待办清单	Copilot 协作产物

2.2 学习闭环

应用最核心的设计是一条「闭环」链路，让每一次学习都能形成可累积的成果：

- ① 语法改错：用户在英语助手输入一个句子，AI 给出正确句子并解释错误原因。
- ② 闭环出题：针对刚才句子里的语法错误，一键生成 3 道填空题。
- ③ 做题批改：AI 逐题批改，判断对错并讲解，最后给出总体鼓励。
- ④ 错题收集：批改中判定为「错」的题，自动进入错题本（去重）。
- ⑤ 重做巩固：在错题本重做，答对后自动移除，记录「成长 +1」。
- ⑥ 全程记录：每一步都写入历史记录，形成成长数据。

2.3 三位一体身份机制

AI 的核心人设并非写死的单一角色，而是通过动态 system prompt 拼装而成。它内置三种身份的行为准则，并依据对用户当前「情绪 + 语言」的实时判断，自动选择最合适的语气：

- 英语老师：纠正语法错误、讲解知识点，先鼓励再纠错。
- 英国朋友：像朋友一样自然聊天，用简单英语交流。
- 心理委员：当检测到用户情绪低落时，先温柔倾听和安慰，再轻轻带一点英语。

2.4 角色人设库

应用内置多套可切换的原创角色人设，用户可在「角色设置」中挑选，也可自定义角色。每个人设都带有「不模仿现有作品」「明确自己是 AI」等安全与版权约束：

角色	定位	人设要点
🐱 **猫娘小棉（默认）**	温柔活泼的英语伙伴	偶尔自然「喵」，先肯定再讲解
🐬 **鲸鱼娘小汐**	沉静、富有想象力	用海洋与潮汐作轻柔比喻

👩‍🏫 **英语老师 Emily**	耐心、专业	先鼓励后纠错，简单英语为主
🔥 **热血动漫主角**	斗志满满	鼓励对方绝不放弃
🤖 **未来 AI 助手**	冷静、高效	简洁清晰，偶尔提到未来趣闻
🎀 **自定义角色**	用户自由定义	用文字描述想扮演的角色

2.5 用户画像体系

画像由「语言习惯」与「情绪状态」两个维度组成，采用轻量规则实时分析，无需额外调用模型，零成本且响应即时：

维度	取值	判断规则
语言习惯	中文为主 / 英语为主 / 中英混合	统计中文字符与英文单词的比例
情绪状态	低落 / 积极 / 平静	命中正负关键词库 + 感叹号密度

三、系统架构

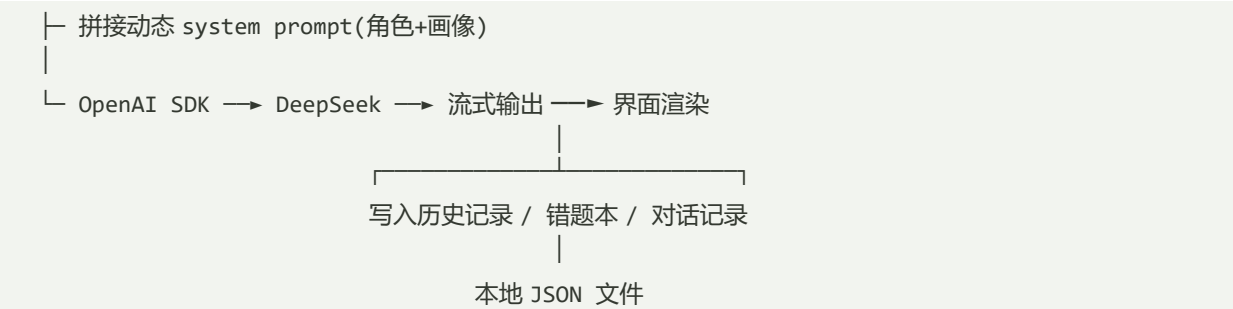
3.1 技术栈

层级	技术选型	说明
前端/界面	Streamlit 1.37.1	纯 Python 快速构建交互界面，含自定义 CSS 动效
大模型	DeepSeek Chat	通过 OpenAI SDK 调用，支持流式输出
客户端	openai 3.39.0	兼容 OpenAI 协议的 Python SDK
图像生成	阿里云百炼·通义万相	可选，生成原创二次元头像
语音识别	Azure Speech	可选，语音输入降级为文字
持久化	本地 JSON 文件	历史/错题/对话记录本地存档

3.2 整体流程

应用采用「单文件、无后端、全会话态」的轻量架构：Streamlit 负责渲染与交互，业务逻辑全部在 app.py 中组织为若干 render_* 视图函数，通过 session_state 维护状态，通过 OpenAI SDK 调用 DeepSeek 大模型。整体数据流如下：

```
用户输入
   │
   ├─ 文本/语音 ──► 画像分析(语言+情绪) ──► 更新 session_state
   │
```



3.3 项目文件结构与职责

文件	职责
app.py	主程序（约 1600 行），含全部业务逻辑、人设库、界面样式
plan.html	学习计划界面的待办清单（独立 HTML，组件嵌入）
requirements.txt	依赖清单
.gitignore	排除隐私数据与密钥文件
README.md	10 分钟复现指南
docs/	技术文档、演示视频脚本、Prompt 快照指南

3.4 数据模型设计

三类数据分别持久化到本地 JSON 文件，结构简洁、可直接阅读与迁移：

```
history.json        # 学习历史
[{ "mode": "语法改错", "text": "...",
   "answer": "...", "created_at": "2026-10-27 22:00" }]

wrongbook.json      # 错题本
[{ "question": "...", "topic": "...",
   "user_answer": "...", "created_at": "..." }]

chat_history.json   # 对话记录
[{ "role": "user", "content": "..." },
 { "role": "assistant", "content": "..." }]
```

四、核心技术实现

4.1 流式对话

为了让回复「像真人打字」一样自然，应用使用 DeepSeek 的流式输出接口，逐块产出内容并实时渲染。核心代码如下：

```
def stream_answer(client, prompt):
    deadline = time.monotonic() + MAX_STREAM_DURATION
    with client.chat.completions.create(
        model=MODEL, messages=[{"role": "user", "content": prompt}],
        temperature=TEMPERATURE, stream=True,
    ) as stream:
        for chunk in stream:
            if time.monotonic() >= deadline:
                raise TimeoutError("AI 回复超过 180 秒")
            delta = chunk.choices[0].delta
            if isinstance(delta.content, str) and delta.content:
                yield delta.content
```

同时设置了「单次请求超时」与「流式总时长上限」双重保护，避免请求无限转圈。

4.2 个性化画像（语言 + 情绪分析）

为了让 AI 「越聊越懂你」，应用在每条用户消息进入对话前，先用轻量规则分析两个维度——语言习惯与情绪状态。该方案无需额外调用模型，零成本且实时：

```
def analyze_user(text):
    # 1) 语言习惯：统计中文字符与英文单词比例
    chinese = len(re.findall(r"[\u4e00-\u9fff]", text))
    english = len(re.findall(r"[A-Za-z]+", text))
    # 2) 情绪：命中关键词库 + 感叹号/波浪号密度
    if any(w in text for w in NEGATIVE_WORDS):
        mood = "低落/需要支持"
    elif any(w in text for w in POSITIVE_WORDS):
        mood = "积极/有活力"
    else:
        mood = "平静"
    return {"language": ..., "mood": mood}
```

分析结果实时更新到用户画像，并展示在聊天界面上方，让「个性化」这个抽象概念变得可见。

4.3 动态 system prompt 拼装

三位一体身份的实现，核心在于「根据画像 + 角色动态拼装 system prompt」。函数 build_friend_system_prompt() 将固定的身份准则、用户选择的角色人设、语言引导、情绪引导四部分拼接成一条完整的系统提示词：

```
base = (
  "你是用户的 AI 英语朋友，可以随时切换三种身份：\n"
  "1) 英语老师：教语法、纠正错误、讲解知识；\n"
  "2) 英国朋友：像英国朋友一样自然聊天；\n"
  "3) 心理委员：当用户情绪不好时，温柔倾听、陪伴和支持。\n"
)
```

```
base += 角色人设 # 例如猫娘小棉 / 英语老师 Emily
base += 语言引导 # 根据用户主要语言决定回复语言
base += 情绪引导 # 根据用户当前情绪决定语气
return base
```

4.4 学习闭环的实现

闭环的关键在于「批改结果的结构化解析」。批改提示词要求模型在结尾单独输出一行 「判定：1 对 2 错 3 错」，应用用正则提取每道题的对错，从而决定哪些题进入错题本：

```
match = re.search(r"判定[:： ]\s*(.+)", full)
if match:
    for num, verdict in re.findall(r"(\d+)\s*(对|错)", match.group(1)):
        if verdict == "错":
            add_wrongbook(题目，知识点，用户答案)
```

这种「约定输出格式 + 正则解析」的方式，让大模型的自由文本输出能够被程序可靠地消费，是实现闭环链路的关键技巧。

4.5 数据持久化

历史记录、错题本、对话记录三类数据分别持久化到本地 JSON 文件，刷新页面不丢失。所有读写都包裹了异常保护，文件写入失败时静默跳过，不影响主流程。隐私数据已通过 .gitignore 排除。

4.6 健壮性处理

所有大模型调用统一封装，针对不同异常给出友好的中文提示：

异常类型	处理方式
APITimeoutError	提示请求超时，请检查网络
AuthenticationError	提示 API Key 无效或过期
RateLimitError	提示频率过高或额度不足
APIConnectionError	提示无法连接服务
未知异常	展示错误详情，便于排查

4.7 关键技术坑：brotli 压缩冲突

开发过程中遇到一个隐蔽的坑：流式回复经常在途中报解压错误。排查后定位到原因是 httpx 2.x 与 brotli 库版本不兼容，导致对 brotli 压缩的响应解压失败。解决方案是在创建客户端时禁用 brotli，强制使用 gzip/deflate：

```
def get_client(api_key):
    return OpenAI(
        api_key=api_key,
        base_url="https://api.deepseek.com",
        default_headers={"Accept-Encoding": "gzip, deflate"},
    )
```

4.8 原创头像生成

AI 头像支持两种方式：默认使用 PIL 程序化绘制一个原创的猫娘头像（零依赖、离线可用）；用户也可通过阿里云百炼「通义万相」生成自定义二次元形象（异步任务 + 轮询下载）。

```
def create_default_catgirl_avatar():
    image = Image.new("RGB", (256, 256))
    draw = ImageDraw.Draw(image)
    draw.polygon(...)    # 猫耳
    draw.ellipse(...)    # 脸型、头发
    draw.ellipse(...)    # 眼睛、腮红、微笑
    return image
```

4.9 语音输入

语音输入采用「浏览器录音 → Azure Speech 识别 → 文本回填」的流程：识别结果先回填到输入框，用户可编辑后再发送；未配置 Azure 服务或 SDK 时自动降级为纯文字输入，不影响核心功能。

```
audio = mic_recorder(format="wav", ...)
text = transcribe_with_azure(audio["bytes"],
        language=voice_language, subscription_key=..., region=...)
```

4.10 界面视觉与动效设计

界面采用「Daylight」设计语言——高对比度、柔和渐变背景、苹果风标题栏、卡片式布局，并通过 CSS 动画（淡入、上浮、弹簧缓动）提升质感：

- 苹果红绿灯标题栏：红黄绿三个圆点 + 已打开界面栈提示。
- 卡片式指标卡：悬浮时上浮 + 阴影加深。
- 毛玻璃侧边栏：背景模糊 + 饱和度提升。
- 无障碍降级：prefers-reduced-motion 时关闭动画。

4.11 性能与体验优化

- @st.cache_resource 缓存 OpenAI 客户端，避免每次点击重复建连。
- 流式输出，首字延迟低，体验接近实时对话。
- 双重超时保护：单次请求超时 + 流式总时长上限。

- 历史记录只保留最近 N 条，控制内存占用与请求体积。

五、Prompt 工程设计

5.1 设计思路

Prompt 工程贯穿整个项目。核心思路是「职责分离」：把「是谁（身份）」「说哪种语言」「用什么语气」拆解成可独立调整的模块，再根据上下文动态组合，而不是写死一段冗长的提示词。

5.2 关键迭代

版本	演进	解决的问题
v1	单一角色：固定「英语老师」人设	能改错，但语气生硬、无情感
v2	加入多种预设角色（猫娘/老师/热血等）	丰富了趣味性，但需手动切换
v3	引入「画像 + 情绪」动态注入	AI 能感知用户情绪，自动变温柔/活泼
v4	三位一体：老师/朋友/心理委员自动切换	实现「懂你」的核心体验

5.3 教学类 Prompt 模板

英语助手等工具型功能采用「结构化模板」，对输出格式做明确约束，保证结果稳定可解析。例如语法改错要求「先给正确句子，再用中文说明改了什么、为什么」；查单词要求「音标、词性释义、三个例句、搭配技巧」四段式输出。

六、AI 协同开发过程

6.1 协作方式

本项目是一次「人机协作」的完整实践，作者在两类 AI 工具的辅助下完成开发：

- DeepSeek：作为「结对编程伙伴」，负责讨论方案、生成核心代码框架、调试疑难 bug（如 brotli 冲突）、优化 Prompt。
- GitHub Copilot：负责界面样式（Daylight 风格 CSS）、学习计划页（plan.html）的生成与代码补全。

值得一提的是，作者是一名零基础的软件工程大一新生。从最初对大学生活的迷茫，到选择用「敢想敢做」来回答自己——报名传智杯、从零学习环境搭建与编程、在 AI 的陪伴下一步步独立完成了这个作品。因此，作品的价值不仅在于实现了多少功能，更在于它真实记录了一个人「从低谷中走出来、最终选择相信自己」的成长过程。

6.2 协作实例

- 方案设计：与 AI 反复讨论「如何让 AI 既像老师又像朋友」，最终确定三位一体架构。
- Bug 排查：流式回复的解压报错，通过多轮对话定位到 httpx/brotli 版本冲突。
- 界面打磨：苹果风标题栏、动效、配色由 Copilot 协作完成。

完整的 AI 协同过程通过对话记录快照作为参赛证据留存（见提交物「AI 对话 Prompt 快照」）。

七、测试与验证

测试项	验证内容	结果
语法改错	输入 I very like AI，验证正确改错与中文解释	通过
闭环出题	改错后一键生成 3 道题并跳转做题本	通过
批改解析	故意答错，验证「判定」行正确解析并入错题本	通过
错题重做	错题本重做答对后自动移除	通过
画像切换	发「好累」→心理委员语气；发「开心」→活泼语气	通过
异常处理	错误 Key / 断网 / 超时的友好提示	通过
数据持久化	刷新页面后历史/错题/对话不丢失	通过

八、部署与运行

8.1 本地运行

```
pip install -r requirements.txt
streamlit run app.py
```

随后在侧边栏填入 DeepSeek API Key 即可使用。详见 README.md 的「10 分钟复现指南」。

8.2 密钥管理

项目不硬编码任何密钥。API Key 仅存于 session 内存，不持久化；如需更优雅的管理，可通过 .streamlit/secrets.toml 配置，该文件同样被 .gitignore 排除。

九、总结与展望

9.1 项目总结

「AI 英语朋友」是作者作为大一新生，第一次借助大模型与 AI 编程助手从零完整交付一个可用的产品。项目实现了三位一体的情感化学习助手，并打通了「改错—出题—批改—错题重做」的完整学习闭环，验证了「轻量规则 + 大模型」这一低成本个性化方案的可行性。

9.2 不足与展望

- 画像分析目前基于规则，未来可引入模型判断，识别更细腻的情绪与学习水平。
- 数据存于本地，未来可接入云端数据库实现多端同步。
- 可增加语音朗读（TTS）、口语对话练习等更沉浸的交互。
- 探索在线部署（Streamlit Community Cloud），降低使用门槛。

十、典型使用场景

以一个备考四级的学生「一天」为例，展示应用如何贯穿学习与情绪陪伴的全过程：

- 早上：在 AI 对话里用简单英语闲聊几句，AI 根据情绪自动切换语气，轻松开口。
- 白天：在英语助手改错一句英文，并针对错误一键生成题目巩固。
- 晚上：在做题本练习，错题自动进入错题本，重做后攻克移除。
- 睡前：打开历史记录回顾成长数字，学习日历看到坚持天数与四级倒计时。

十一、安全与隐私设计

- API Key 不硬编码：仅存于 session 内存，不持久化，关闭即失效。
- 隐私数据隔离：history / wrongbook / chat 三类 JSON 均通过 .gitignore 排除。
- 密钥文件保护：.streamlit/secrets.toml 同样被排除在版本库之外。
- 外部服务可选：语音、图像生成等均需用户主动配置 Key，未配置时优雅降级。